

BookingPartner.Ik

12-week backend schedule + daily do-list

Start: Friday, 18 September 2026 | Soft-launch target: Thursday, 10 December 2026

What this PDF does

It turns the backend roadmap into small daily jobs. Finish one day's checks before moving to the next. The first two days are separated as TODAY and TOMORROW so you can start immediately.

Simple mental model

Frontend	What the customer sees: search, seat map, forms, dashboards.
Backend	The hidden worker: checks seats, payments, permissions, bookings, refunds, and earnings.
Database	The permanent record book: users, buses, schedules, seats, bookings, payments, and history.

The basic flow

Seat example

Customer clicks a seat -> backend checks the database -> backend locks the seat -> database saves the lock -> Pusher tells other browsers -> frontend shows the new seat state.

TODAY - Friday, 18 September 2026

TODAY

Day 1: Install local database

Friday, 18 September 2026

- ☐ Install PostgreSQL on the laptop.
- ☐ Create a local database named bookingpartner_dev.
- ☐ Put the local DATABASE_URL and other development keys in .env.local.
- ☐ Run `npx prisma migrate dev` and `npx prisma generate`.
- ☐ Open pgAdmin and confirm the project tables are visible.
- ☐ Start `npm run dev` and make sure the website still opens normally.

Done when: the app runs locally and PostgreSQL shows the expected project tables.

Suggested order

- ☐ First install PostgreSQL and remember the password you set.
- ☐ Then create bookingpartner_dev in pgAdmin.
- ☐ Then fill .env.local with development values only.
- ☐ Run the Prisma commands, then open pgAdmin and visually confirm tables exist.
- ☐ Finally run the website and make sure today's setup did not break the frontend.

Do not do today

Do not connect the local app to a production customer database. Do not put any secret key in browser code or commit .env.local to Git.

TOMORROW - Saturday, 19 September 2026

TOMORROW

Day 2: Connect the app to users and login

Saturday, 19 September 2026

- ☐ Create lib/prisma.ts so the app reuses one safe database connection.
- ☐ Create lib/auth.ts with the login rules and session settings.
- ☐ Set up `/api/auth/[...nextauth]/route.ts`.
- ☐ Create one test user in the local database.
- ☐ Log in with that user and confirm the session knows the user's ID and role.
- ☐ Log out and confirm the session disappears.

Done when: a test user can log in, stay logged in, and log out without errors.

What each tool is doing

Next.js	Your website plus the server-side route.ts files.
PostgreSQL	The database that remembers users, buses, seats, bookings, and money records.
Prisma	The translator between your TypeScript code and PostgreSQL.
Pusher	The live messenger that updates other seat maps without refresh.
PayHere	The payment service. Your backend trusts a payment only after verifying PayHere's server message.
Resend	Sends booking and account emails.
SMS provider	Sends OTPs and booking updates to phones.
Vercel	Runs the Next.js app in production.
Railway	Hosts the production PostgreSQL database.

One-time laptop setup checklist

- ☐ Node.js works: `node --version` shows a version number.
- ☐ PostgreSQL is installed locally.
- ☐ pgAdmin can open `bookingpartner_dev`.
- ☐ `.env.local` contains development-only keys.
- ☐ `npm install` finishes successfully.
- ☐ `npx prisma migrate dev --name init` finishes successfully when appropriate for this project state.
- ☐ `npx prisma generate` finishes successfully.
- ☐ `npm run dev` opens the app at `localhost:3000`.

Important

If the project already has migrations, do not blindly create a second 'init' migration. Use the existing migration history and run the Prisma command appropriate for the current project. The goal is a clean local schema, not duplicate migrations.

Five rules to keep visible while you build

1. Check the person first

Private API routes must check the session. No login -> 401. Wrong role -> 403. Ownership must be checked in the database.

2. Trust PayHere only after verification

A browser saying 'payment success' is not proof. Only the verified PayHere server message can confirm a paid booking.

3. Seat changes must be one safe database action

When two people click the same seat, only one request may win. Use an atomic conditional update/transaction and database constraints.

4. Secrets stay on the server

Database URLs, payment secrets, email keys, and cron secrets never go into frontend code or public Git.

5. Build one endpoint, then test it

Finish the normal case, bad input, no-login case, wrong-role case, and wrong-owner case before moving on.

Week result: Local database, login, page protection, live updates, email, and PayHere sandbox are connected.

TODAY**Day 1: Install local database**

Friday, 18 September 2026

- ☐ Install PostgreSQL on the laptop.
- ☐ Create a local database named bookingpartner_dev.
- ☐ Put the local DATABASE_URL and other development keys in .env.local.
- ☐ Run npx prisma migrate dev and npx prisma generate.
- ☐ Open pgAdmin and confirm the project tables are visible.
- ☐ Start npm run dev and make sure the website still opens normally.

Done when: the app runs locally and PostgreSQL shows the expected project tables.

TOMORROW**Day 2: Connect the app to users and login**

Saturday, 19 September 2026

- ☐ Create lib/prisma.ts so the app reuses one safe database connection.
- ☐ Create lib/auth.ts with the login rules and session settings.
- ☐ Set up /api/auth/[...nextauth]/route.ts.
- ☐ Create one test user in the local database.
- ☐ Log in with that user and confirm the session knows the user's ID and role.
- ☐ Log out and confirm the session disappears.

Done when: a test user can log in, stay logged in, and log out without errors.

Day 3: Protect private pages

Sunday, 20 September 2026

- ☐ Create src/middleware.ts.
- ☐ Block logged-out visitors from owner, management, and admin pages.
- ☐ Allow each role to enter only its own area.
- ☐ Send the wrong role to a safe page instead of showing private data.
- ☐ Test owner, management, admin, and normal customer accounts.

Done when: every private page is blocked for the wrong person before page content loads.

Day 4: Set up live seat updates

Monday, 21 September 2026

- ☐ Create a Pusher account and development app.
- ☐ Add the Pusher keys to .env.local.
- ☐ Install pusher and pusher-js.
- ☐ Create lib/pusher-server.ts and lib/pusher-client.ts.
- ☐ Make a tiny test event from the backend.
- ☐ Open two browser tabs and confirm the second tab receives the event without refresh.

Done when: two open browser tabs can receive a live event instantly.

Day 5: Set up booking emails

Tuesday, 22 September 2026

- ☐ Create a Resend account and development API key.
- ☐ Add RESEND_API_KEY to .env.local.
- ☐ Create lib/email.ts.
- ☐ Write a simple sendBookingConfirmation() function.
- ☐ Send one test email to an address you can check.
- ☐ Confirm the email arrives and the error is handled if sending fails.

Done when: a test booking email arrives successfully and failure does not crash the app.

Day 6: Set up PayHere sandbox

Wednesday, 23 September 2026

- ☐ Create or open the PayHere sandbox merchant account.
- ☐ Add sandbox merchant ID, secret, and sandbox flag to .env.local.
- ☐ Create lib/payhere.ts.
- ☐ Write generatePayHereData() for the payment form values.
- ☐ Keep every secret on the server only.

Done when: the app can prepare a sandbox payment request without exposing the PayHere secret.

Day 7: Verify PayHere messages

Thursday, 24 September 2026

- ☐ Write verifyPayHereWebhook().
- ☐ Check the PayHere signature before trusting any payment result.
- ☐ Create test examples for valid, invalid, and missing signatures.
- ☐ Return a safe error for bad payment messages.
- ☐ Write a short note in the code explaining what must never be skipped.

Done when: fake or damaged payment messages are rejected and a valid test message passes.

Week result: Customer and owner sign-up, verification, login, OTP, and password recovery work end-to-end.

Day 8: Customer registration API

Friday, 25 September 2026

- ☐ Create POST /api/v1/auth/register.
- ☐ Check that name, email, phone, and password are present and sensible.
- ☐ Reject an email that already exists.
- ☐ Hash the password before saving it.
- ☐ Create the new user with the normal customer role.
- ☐ Create a verification token and send the verification email.

Done when: a new customer can register and receives a verification email.

Day 9: Finish registration and login checks

Saturday, 26 September 2026

- ☐ Test good registration, duplicate email, weak or missing fields, and server errors.
- ☐ Make sure the API never returns the password hash.
- ☐ Confirm NextAuth login works with the registered test user.
- ☐ Return simple, useful error messages the frontend can show.
- ☐ Add basic logs for failed registrations without logging secrets.

Done when: registration and login work together and bad input is safely rejected.

Day 10: Email verification

Sunday, 27 September 2026

- ☐ Create GET /api/v1/auth/verify-email?token=xxx.
- ☐ Find the user linked to the token.
- ☐ Reject expired or already-used tokens.
- ☐ Set is_verified to true after a valid check.
- ☐ Remove or invalidate the used token.
- ☐ Send the user back to login with a clear success message.

Done when: one verification link works once and invalid links do not verify an account.

Day 11: Owner OTP: send and verify

Monday, 28 September 2026

- ☐ Create POST /api/v1/owner-auth/send-otp.
- ☐ Generate a short-lived OTP and store it safely.
- ☐ Send the OTP through the chosen SMS provider or development test method.
- ☐ Create POST /api/v1/owner-auth/verify-otp.
- ☐ Limit repeated tries and expire old OTPs.

Done when: the correct OTP works within its time limit and wrong or old OTPs fail.

Day 12: Owner registration

Tuesday, 29 September 2026

- ☐ Create POST /api/v1/owner-auth/register.
- ☐ Require a successfully verified OTP before creating an owner account.
- ☐ Save owner/company details and the correct owner role.
- ☐ Prevent the same email or phone from creating duplicate accounts.
- ☐ Test the full owner registration from start to finish.

Done when: a verified person can create an owner account and an unverified person cannot.

Day 13: Forgot and reset password

Wednesday, 30 September 2026

- ☐ Create POST /api/v1/auth/forgot-password.
- ☐ Create a short-lived reset token without revealing whether an email exists.
- ☐ Send the reset email.
- ☐ Create POST /api/v1/auth/reset-password.
- ☐ Check token validity, hash the new password, and invalidate the token.

Done when: a test user can request a reset, set a new password, and the old password stops working.

Day 14: Auth test day

Thursday, 01 October 2026

- ☐ Run customer register, verify, login, logout, forgot password, and reset password.
- ☐ Run owner OTP and owner registration.
- ☐ Try wrong roles and logged-out access.
- ☐ Fix any 500 errors and replace them with safe 4xx responses where appropriate.
- ☐ Write a small checklist of auth routes that now pass.

Done when: all authentication flows pass end-to-end without manual database fixing.

Week result: Public search basics and owner bus/seat-layout tools work with real database data.

Day 15: Cities and routes

Friday, 02 October 2026

- ☐ Create GET /api/v1/cities returning active cities only.
- ☐ Create GET /api/v1/routes with from-city and to-city details.
- ☐ Sort dropdown results in a useful order.
- ☐ Hide inactive records from public results.
- ☐ Connect one frontend dropdown to real city data.

Done when: the frontend can load real cities and routes from the database.

Day 16: Public schedule search

Saturday, 03 October 2026

- ☐ Create GET /api/v1/search?from=x&to=y&date=z.
- ☐ Validate the route and date values.
- ☐ Return matching schedules with bus, departure time, price, and seats left.
- ☐ Do not return cancelled or hidden schedules.
- ☐ Test no-results, one-result, and many-results searches.

Done when: a real search from the homepage returns the correct schedule cards.

Day 17: Owner bus list and add

Sunday, 04 October 2026

- ☐ Create GET /api/v1/owner/buses for the signed-in owner only.
- ☐ Create POST /api/v1/owner/buses.
- ☐ Validate required bus details before saving.
- ☐ Never accept an owner ID from the browser as proof of ownership.
- ☐ Test owner A cannot see owner B's buses.

Done when: an owner can list and add only their own buses.

Day 18: Owner bus details and edit

Monday, 05 October 2026

- ☐ Create GET /api/v1/owner/buses/[id].
- ☐ Create PATCH /api/v1/owner/buses/[id].
- ☐ Check ownership using both bus ID and signed-in owner ID.
- ☐ Allow only editable fields to change.
- ☐ Return 404 for a bus the owner does not own.

Done when: bus details can be viewed and changed only by the correct owner.

Day 19: Seat layout read and save

Tuesday, 06 October 2026

- ☐ Create GET /api/v1/buses/[id]/seats.
- ☐ Create POST /api/v1/buses/[id]/seats for authorized owners.
- ☐ Save seat position, label, type, and availability rules.
- ☐ Reject duplicate seat labels inside the same bus.
- ☐ Test a normal 2+2 layout.

Done when: one bus seat layout can be saved, reloaded, and shown exactly the same.

Day 20: Seat number generator

Wednesday, 07 October 2026

- ☐ Create the seat-number generation logic.
- ☐ Test every supported bus layout type.
- ☐ Keep aisle or non-seat positions out of the sellable seat list.
- ☐ Check numbering is stable after saving and reloading.
- ☐ Handle special seats such as driver area or blocked positions.

Done when: all supported layouts generate unique, predictable seat numbers.

Day 21: Buses and seats test day

Thursday, 08 October 2026

- ☐ Add several test buses with different layouts.
- ☐ Edit one bus and confirm unrelated fields stay unchanged.
- ☐ Try to access another owner's bus by changing the URL ID.
- ☐ Save and reload every seat layout type.
- ☐ Fix any ownership or duplicate-seat problems.

Done when: bus and seat APIs pass normal and security tests.

Week result: Owners can create, repeat, view, move, cancel, and export trip manifests safely.

Day 22: Create schedule

Friday, 09 October 2026

- ☐ Create POST /api/v1/owner/schedules.
- ☐ Check the bus belongs to the owner.
- ☐ Validate route, date, departure time, price, and required fields.
- ☐ Save the schedule.
- ☐ After saving, create one schedule-seat row for every sellable bus seat.

Done when: creating one schedule automatically creates its complete seat inventory.

Day 23: List and view schedules

Saturday, 10 October 2026

- ☐ Create GET /api/v1/owner/schedules.
- ☐ Add filters for bus, date, and status.
- ☐ Create GET /api/v1/owner/schedules/[id].
- ☐ Return live counts such as available, locked, and booked seats.
- ☐ Keep results limited to the signed-in owner.

Done when: the owner schedule page can load real lists and one full schedule.

Day 24: Repeat schedules

Sunday, 11 October 2026

- ☐ Add repeat options such as daily and weekly.
- ☐ Create the required future schedule rows for the chosen repeat range.
- ☐ Give repeated rows the same repeat_group_id.
- ☐ Generate a fresh set of schedule seats for every repeated trip.
- ☐ Use one database transaction so a partial repeat set is not left behind.

Done when: a repeat request creates the correct dates and every trip has its own seats.

Day 25: Schedule conflict warning

Monday, 12 October 2026

- ☐ Check whether the same bus already has another schedule at the same or overlapping time.
- ☐ Return a warning with the conflicting trip details.
- ☐ Do not silently overwrite an existing schedule.
- ☐ Keep the warning separate from hard validation errors.
- ☐ Test clear, exact-conflict, and close-time cases.

Done when: the owner is clearly warned before creating a conflicting bus schedule.

Day 26: Move departure time

Tuesday, 13 October 2026

- ☐ Create PATCH /api/v1/owner/schedules/[id]/update-time.
- ☐ Allow only the correct owner or permitted manager.
- ☐ Apply the business rule for allowed time changes.
- ☐ Save the new departure time in one safe update.
- ☐ Queue notifications for passengers already confirmed on that trip.

Done when: an allowed time change updates once and all affected confirmed passengers are identified.

Day 27: Cancel schedule safely

Wednesday, 14 October 2026

- ☐ Create POST /api/v1/owner/schedules/[id]/cancel.
- ☐ If no bookings exist, cancel the schedule directly.
- ☐ If bookings exist, apply the cancellation-time business rule.
- ☐ For eligible cancellations, update bookings and create wallet refund records.
- ☐ Notify affected passengers and make seats unavailable for further sale.

Done when: the correct cancellation path is used for empty and booked schedules.

Day 28: Passenger manifest PDF

Thursday, 15 October 2026

- ☐ Create GET /api/v1/owner/schedules/[id]/manifest.
- ☐ Return passenger name, seat, contact, booking reference, and status as permitted.
- ☐ Keep the manifest limited to the correct owner/manager.
- ☐ Generate a clean printable PDF.
- ☐ Test a schedule with zero, a few, and many passengers.

Done when: the owner can download a readable manifest PDF for one authorized schedule.

Week result: Seat state is safe under two-user races, expires automatically, and updates browsers live.

Day 29: Atomic seat lock

Friday, 16 October 2026

- ☐ Create POST /api/v1/seats/lock.
- ☐ Check the user/session and validate schedule and seat IDs.
- ☐ Change the seat only if it is still available in the same database action.
- ☐ Save who locked it and the lock expiry time.
- ☐ Return 409 if another person got the seat first.
- ☐ Send a Pusher event after a successful lock.

Done when: two people racing for the same seat cannot both receive a successful lock.

Day 30: Seat release

Saturday, 17 October 2026

- ☐ Create POST /api/v1/seats/release.
- ☐ Release only a seat locked by the current user or the authorized flow.
- ☐ Do not release an already confirmed booking.
- ☐ Clear locked_by and locked_until.
- ☐ Send a Pusher event so other browsers see the seat become available.

Done when: a user can release their own temporary lock and cannot release someone else's.

Day 31: Expired-lock cleanup

Sunday, 18 October 2026

- ☐ Create POST /api/cron/release-locks.
- ☐ Require CRON_SECRET before doing any work.
- ☐ Find temporary locks whose expiry time has passed.
- ☐ Return them to available status safely.
- ☐ Record how many were released.
- ☐ Send live updates for affected schedules.

Done when: an expired test lock becomes available after the cleanup route runs.

Day 32: Schedule the cleanup job

Monday, 19 October 2026

- ☐ Add the Vercel cron configuration for the release-lock route.
- ☐ Use the production CRON_SECRET through environment variables.
- ☐ Keep the job safe if it runs twice close together.
- ☐ Test the route manually with the correct and wrong secret.
- ☐ Check logs without printing private user data.

Done when: the cleanup job can run repeatedly without double-changing or exposing data.

Day 33: Seat-map read API

Tuesday, 20 October 2026

- ☐ Create GET /api/v1/seats/[scheduleId].
- ☐ Return each seat's display label and current safe status.
- ☐ Treat expired locks as available or clean them before responding.
- ☐ Do not expose another user's private lock details.
- ☐ Connect the initial seat-map page load to this API.

Done when: opening a seat map shows the correct current status before live events begin.

Day 34: Real-time two-browser test

Wednesday, 21 October 2026

- ☐ Open the same trip in two separate browsers or devices.
- ☐ Lock a seat in browser A and watch browser B change without refresh.
- ☐ Release it and confirm both screens update.
- ☐ Confirm a confirmed booking becomes permanently unavailable.
- ☐ Test reconnecting after temporarily losing the live connection.

Done when: both browsers stay in sync for lock, release, and booked events.

Day 35: Seat-lock stress and edge cases

Thursday, 22 October 2026

- ☐ Send near-simultaneous lock requests for the same seat.
- ☐ Test expired locks, double-release, invalid seat, invalid schedule, and logged-out user.
- ☐ Confirm database rules prevent duplicate confirmed use of the same schedule seat.
- ☐ Check every failure returns a clear status code.
- ☐ Fix any path that can leave a seat stuck.

Done when: all seat-lock edge cases finish in one valid final seat state.

Week result: A customer can create, pay, confirm, view, cancel, refund, and download a ticket.

Day 36: Create pending booking

Friday, 23 October 2026

- ☐ Create POST /api/v1/bookings.
- ☐ Check the selected seat is currently locked for this customer.
- ☐ Copy passenger, fare, policy, and trip details needed for a permanent booking record.
- ☐ Create the booking with pending status.
- ☐ Generate PayHere sandbox form values and return only safe data to the browser.

Done when: a locked seat can create exactly one pending booking ready for sandbox payment.

Day 37: PayHere payment webhook

Saturday, 24 October 2026

- ☐ Create POST /api/v1/payments/webhook.
- ☐ Verify the PayHere signature before changing anything.
- ☐ Match the payment to the correct pending booking.
- ☐ Make the handler safe if PayHere sends the same success message more than once.
- ☐ On valid success, confirm booking and seat and create the payout record.
- ☐ Send confirmation email/SMS and Pusher booked event.

Done when: one valid payment confirms once, duplicates do nothing harmful, and fake messages fail.

Day 38: Wallet payment support

Sunday, 25 October 2026

- ☐ Calculate how much wallet credit the user may apply.
- ☐ Reduce only the remaining external payment amount.
- ☐ Create a wallet transaction record for every wallet movement.
- ☐ Update wallet balance in the same safe database transaction.
- ☐ Prevent a balance from going below zero.

Done when: wallet and external payment amounts always add up to the booking total.

Day 39: Customer booking list

Monday, 26 October 2026

- ☐ Create GET /api/v1/bookings for the current customer only.
- ☐ Return useful trip summary, seat, status, and booking reference.
- ☐ Add basic sorting and pagination if needed.
- ☐ Hide internal payout and security fields.
- ☐ Connect the customer dashboard to real data.

Done when: a customer sees only their own real bookings in the dashboard.

Day 40: Secure booking detail

Tuesday, 27 October 2026

- ☐ Create GET /api/v1/bookings/[id].
- ☐ Query using both booking ID and current customer ID.
- ☐ Return 404 for another customer's booking instead of leaking that it exists.
- ☐ Return passenger and trip details needed by the detail page.
- ☐ Try changing the booking ID manually as a security test.

Done when: customer A cannot read customer B's booking even with the correct booking ID.

Day 41: Customer cancellation and refund

Wednesday, 28 October 2026

- ☐ Create POST /api/v1/bookings/[id]/cancel.
- ☐ Check ownership and current booking status.
- ☐ Read the saved policy_snapshot instead of today's policy.
- ☐ Calculate eligible refund and credit the wallet with a recorded transaction.
- ☐ Update the booking, release the seat when allowed, notify the user, and broadcast the seat change.

Done when: a cancellable booking changes once, receives the correct saved-policy refund, and frees the seat.

Day 42: Ticket PDF and QR

Thursday, 29 October 2026

- ☐ Create GET /api/v1/bookings/[id]/ticket.
- ☐ Check the booking belongs to the signed-in customer.
- ☐ Create a ticket PDF with booking reference, route, date/time, seat, passenger, and operator.
- ☐ Add a QR value that identifies the booking safely.
- ☐ Test opening, printing, and scanning the ticket.

Done when: a confirmed customer can download a readable ticket whose QR points to the correct booking reference.

Week result: Owner operations use real data and employee permissions are enforced on the backend.

Day 43: Owner earnings summary

Friday, 30 October 2026

- ☐ Create GET /api/v1/owner/earnings.
- ☐ Calculate totals from the owner's real completed/eligible bookings and payouts.
- ☐ Separate pending, sent, and other defined payout states.
- ☐ Group useful numbers by trip for the earnings cards.
- ☐ Confirm owner A cannot include owner B's money.

Done when: the owner earnings page matches hand-checked test bookings and payout records.

Day 44: Owner payout list

Saturday, 31 October 2026

- ☐ Create GET /api/v1/owner/payouts.
- ☐ Return trip, amount, status, and relevant dates.
- ☐ Keep the result restricted to the signed-in owner.
- ☐ Add pagination or date filtering if the page needs it.
- ☐ Connect the payout page to real results.

Done when: the payout page shows the same totals and states as the database for that owner.

Day 45: Invite and list employees

Sunday, 01 November 2026

- ☐ Create POST /api/v1/owner/employees/invite.
- ☐ Create a safe invitation token and send the invitation email.
- ☐ Create GET /api/v1/owner/employees.
- ☐ Show active and pending employees without exposing secrets.
- ☐ Prevent an owner from managing another owner's staff.

Done when: an owner can invite a worker and see the invitation in their own staff list.

Day 46: Employee permissions and assignments

Monday, 02 November 2026

- ☐ Create POST /api/v1/owner/employees/assign.
- ☐ Create PATCH /api/v1/owner/employees/[id].
- ☐ Create DELETE /api/v1/owner/employees/[id] as a safe deactivate action.
- ☐ Link employees only to buses belonging to that owner.
- ☐ Enforce permissions on the backend, not just by hiding buttons.

Done when: an employee can access only the buses and actions the owner explicitly allowed.

Day 47: Owner company and bank settings

Tuesday, 03 November 2026

- ☐ Create PATCH /api/v1/owner/settings/company.
- ☐ Create POST /api/v1/owner/settings/bank/request.
- ☐ Validate company fields and bank-change request fields.
- ☐ Do not expose stored sensitive bank details in normal API responses.
- ☐ Record important change requests for later admin review.

Done when: company settings save correctly and bank changes follow a controlled request flow.

Day 48: Policy, notifications, and password

Wednesday, 04 November 2026

- ☐ Create PATCH /api/v1/owner/settings/policy.
- ☐ Create PATCH /api/v1/owner/settings/notifications.
- ☐ Create POST /api/v1/owner/settings/password.
- ☐ Require the current password or appropriate re-authentication before changing it.
- ☐ Keep policy changes from silently changing old booking snapshots.

Done when: the owner can update settings safely and old bookings keep their original policy copy.

Day 49: Manual owner booking

Thursday, 05 November 2026

- ☐ Create POST /api/v1/owner/bookings.
- ☐ Check bus/schedule ownership and seat availability.
- ☐ Create the booking as manual with no online payment or platform commission where required.
- ☐ Confirm the seat in the same safe operation.
- ☐ Send normal passenger confirmation if contact details are present.

Done when: an owner can create a walk-in/manual booking without creating a fake online payment.

Week result: Admin review, payout, report, and analytics endpoints are protected and auditable.

Day 50: Admin owner list

Friday, 06 November 2026

- ☐ Create GET /api/v1/admin/owners.
- ☐ Require admin role before any query.
- ☐ Add pagination, search, and useful approval status fields.
- ☐ Return only fields the admin screen needs.
- ☐ Test a normal owner and management user cannot access it.

Done when: only admins can load the owner review list.

Day 51: Admin owner actions and audit log

Saturday, 07 November 2026

- ☐ Create PATCH /api/v1/admin/owners/[id].
- ☐ Support the defined approve, reject, and suspend actions.
- ☐ Require a reason where the business flow needs one.
- ☐ Write each action to audit_logs with actor, target, action, and time.
- ☐ Prevent an invalid state change.

Done when: every owner status change is authorized and leaves a clear audit record.

Day 52: Admin bus verification

Sunday, 08 November 2026

- ☐ Create GET /api/v1/admin/buses for buses needing review.
- ☐ Create PATCH /api/v1/admin/buses/[id].
- ☐ Support approve/reject with reason where needed.
- ☐ Write verification actions to the audit log.
- ☐ Confirm rejected or unverified buses follow the correct public visibility rule.

Done when: admins can review buses and every decision is traceable.

Day 53: Admin payout management

Monday, 09 November 2026

- ☐ Create GET /api/v1/admin/payouts.
- ☐ Create PATCH /api/v1/admin/payouts/[id] for defined release/withhold actions.
- ☐ Create POST /api/v1/admin/payouts/release-all with strong checks.
- ☐ Make payout actions idempotent so a repeated click cannot pay twice.
- ☐ Audit every manual payout decision.

Done when: a payout can change state once, cannot be duplicated, and is recorded in audit logs.

Day 54: Admin reports list and reply

Tuesday, 10 November 2026

- ☐ Create GET /api/v1/admin/reports.
- ☐ Add filters for status or report type.
- ☐ Create POST /api/v1/admin/reports/[id]/reply.
- ☐ Save the reply and notify the reporting user if required.
- ☐ Keep private admin notes separate from user-visible replies.

Done when: an admin can read a report and send a controlled reply without exposing private notes.

Day 55: Admin report status

Wednesday, 11 November 2026

- ☐ Create PATCH /api/v1/admin/reports/[id]/status.
- ☐ Allow only defined status changes.
- ☐ Record who changed the status and when.
- ☐ Make the admin UI reload the updated state.
- ☐ Test unauthorized access and invalid report IDs.

Done when: report status updates are authorized, validated, and auditable.

Day 56: Admin analytics

Thursday, 12 November 2026

- ☐ Create GET /api/v1/admin/analytics.
- ☐ Return bookings per day, revenue breakdown, top routes, and other required dashboard numbers.
- ☐ Define exactly which booking statuses count in each number.
- ☐ Use date filters consistently.
- ☐ Compare a small test dataset by hand to the API output.

Done when: analytics numbers match a hand-calculated sample and use clear definitions.

Week result: Management users, ticket transfers, reviews, and notification center are complete.

Day 57: Management schedule APIs

Friday, 13 November 2026

- ☐ Create GET /api/v1/management/schedules.
- ☐ Create POST /api/v1/management/schedules.
- ☐ Require management role and link every result to the correct owner/company.
- ☐ Apply the employee/management permissions already defined.
- ☐ Test a manager from owner A cannot see owner B's schedules.

Done when: a manager can list and create only schedules inside their assigned owner account.

Day 58: Management bookings and buses

Saturday, 14 November 2026

- ☐ Create GET /api/v1/management/bookings.
- ☐ Create POST /api/v1/management/bookings for walk-ins.
- ☐ Create GET /api/v1/management/buses.
- ☐ Reuse the same seat safety rules as owner/customer bookings.
- ☐ Check every query is scoped to the manager's owner and allowed buses.

Done when: management users can do their job without crossing into another operator's data.

Day 59: Ticket transfer request and preview

Sunday, 15 November 2026

- ☐ Create POST /api/v1/transfers.
- ☐ Check the booking is transferable under the business rules.
- ☐ Create a short-lived transfer code linked to the booking.
- ☐ Create GET /api/v1/transfers/[code] that returns only safe preview details.
- ☐ Do not reveal private account data before acceptance.

Done when: a valid booking can create a limited transfer code and the receiver sees a safe preview.

Day 60: Accept, decline, and expire transfers

Monday, 16 November 2026

- ☐ Create POST /api/v1/transfers/[code]/accept.
- ☐ Create POST /api/v1/transfers/[code]/decline.
- ☐ Make acceptance change ownership only once in a transaction.
- ☐ Create the cleanup job for pending transfers older than 2 hours.
- ☐ Notify both sides about accept, decline, or expiry.

Done when: a transfer reaches exactly one final state and cannot be accepted twice.

Day 61: Create reviews

Tuesday, 17 November 2026

- ☐ Create POST /api/v1/reviews.
- ☐ Allow reviews only for eligible completed travel.
- ☐ Require the review to be linked to the customer's own booking.
- ☐ Prevent duplicate review submissions for the same eligible booking if that is the rule.
- ☐ Validate rating and text length.

Done when: only a real eligible passenger can create a valid review.

Day 62: Read ratings

Wednesday, 18 November 2026

- ☐ Create GET /api/v1/reviews/[busId].
- ☐ Return published reviews and the calculated average rating.
- ☐ Define which reviews are included in the average.
- ☐ Paginate long review lists.
- ☐ Compare the average with a small hand-calculated sample.

Done when: the bus page shows the correct review list and average.

Day 63: Notifications

Thursday, 19 November 2026

- ☐ Create GET /api/v1/notifications for the signed-in user.
- ☐ Create POST /api/v1/notifications/read.
- ☐ Return only that user's notifications.
- ☐ Connect booking, cancellation, schedule change, transfer, and other major actions to notification creation.
- ☐ Test read/unread counts in the frontend.

Done when: the notification center shows only the correct user's events and read state updates correctly.

Week result: Security, payments, live seats, cron jobs, and the real frontend are tested together.

Day 64: API security sweep - customer routes

Friday, 20 November 2026

- ☐ List every customer API endpoint in one test sheet.
- ☐ Check logged-out requests return 401 where required.
- ☐ Check customer A cannot read or change customer B's data by guessing IDs.
- ☐ Check validation for missing, invalid, and unexpected fields.
- ☐ Fix any endpoint that trusts a customer ID sent by the browser.

Done when: all customer endpoints pass session, ownership, and input checks.

Day 65: API security sweep - staff routes

Saturday, 21 November 2026

- ☐ List every owner, management, and admin API endpoint.
- ☐ Check wrong roles return 403.
- ☐ Check owner/management records are always scoped to the correct operator.
- ☐ Check destructive actions leave audit logs where required.
- ☐ Review logs to ensure secrets, passwords, and full payment data are never printed.

Done when: role boundaries hold across all staff endpoints.

Day 66: Real PayHere sandbox tests

Sunday, 22 November 2026

- ☐ Run a complete successful sandbox payment.
- ☐ Test failed/cancelled payment paths.
- ☐ Send a duplicate success callback and confirm it does not duplicate booking or payout records.
- ☐ Send an invalid signature and confirm nothing changes.
- ☐ Compare the booking, seat, payment, wallet, and payout rows after each test.

Done when: all sandbox payment outcomes leave the database in the expected state.

Day 67: Seat and cron reliability tests

Monday, 23 November 2026

- ☐ Run many near-simultaneous seat lock attempts.
- ☐ Create expired locks and run the cron endpoint.
- ☐ Run the same cron twice and confirm the second run is harmless.
- ☐ Test transfer expiry and any other scheduled cleanup jobs.
- ☐ Fix stuck-state or double-processing problems.

Done when: concurrent actions and repeated cleanup jobs remain safe.

Day 68: Connect frontend to real APIs - part 1

Tuesday, 24 November 2026

- ☐ Replace mock data on search, seat map, authentication, and booking pages.
- ☐ Use loading, empty, success, and error states.
- ☐ Match frontend field names to the real API response shape.
- ☐ Remove any frontend-only fake availability logic.
- ☐ Test on desktop and mobile sizes.

Done when: the core customer journey uses real backend data instead of mocks.

Day 69: Connect frontend to real APIs - part 2

Wednesday, 25 November 2026

- ☐ Replace mock data on owner, management, and admin pages.
- ☐ Connect earnings, payouts, buses, schedules, employees, settings, reports, and analytics.
- ☐ Handle 401 and 403 responses cleanly.
- ☐ Fix missing fields or response-shape mismatches.
- ☐ Remove temporary hard-coded demo values that could appear in production.

Done when: all main dashboards read and change real authorized data.

Day 70: Full end-to-end test

Thursday, 26 November 2026

- ☐ Start with a clean test customer.
- ☐ Register, verify, log in, search, choose a seat, pay in sandbox, and receive confirmation.
- ☐ Confirm the seat is booked on a second browser.
- ☐ Confirm email/SMS, customer dashboard booking, ticket PDF, owner earnings/payout data, and notifications.
- ☐ Write down every bug found and fix launch-blocking issues first.

Done when: one complete customer journey works from registration to confirmed ticket without manual database edits.

Week result: Vercel and Railway are configured, migrated, and tested with production environment settings.

Day 71: Connect GitHub and Vercel

Friday, 27 November 2026

- ☐ Connect the production GitHub repository to Vercel.
- ☐ Set the production branch and deployment rules.
- ☐ Deploy once and confirm the app builds without local-only files.

Done when: the production URL loads a successful build.

Day 72: Create Railway production database

Saturday, 28 November 2026

- ☐ Create the production PostgreSQL service on Railway.
- ☐ Create a strong production database password/URL.
- ☐ Keep the value only in secure environment settings.

Done when: the production database is running and not exposed in frontend code.

Day 73: Run production migrations

Sunday, 29 November 2026

- ☐ Back up first if this is not an empty database.
- ☐ Run `npx prisma migrate deploy` against production.
- ☐ Run `npx prisma generate` during the build as required.
- ☐ Check expected tables and indexes.

Done when: production schema matches the tested code version.

Day 74: Set production environment variables

Monday, 30 November 2026

- ☐ Add database, auth, Pusher, PayHere, Resend, SMS, app URL, and cron secrets in Vercel.
- ☐ Use production keys where ready and sandbox keys only where intentionally testing.
- ☐ Redeploy after environment changes.

Done when: the deployed app can connect to every required service.

Day 75: Production smoke test

Tuesday, 01 December 2026

- ☐ Test login, search, seat view, owner pages, and admin access.
- ☐ Trigger a safe test email and live event.
- ☐ Check server logs for errors without exposing secrets.

Done when: all key pages and service connections work on the real URL.

Day 76: Production payment dry run

Wednesday, 02 December 2026

- ☐ Use the approved test/sandbox path first.
- ☐ Confirm webhook URL is reachable from PayHere.
- ☐ Check one payment path from browser to callback to booking state.

Done when: the end-to-end payment path works on the deployed app.

Day 77: Week 11 review

Thursday, 03 December 2026

- ☐ Review errors and performance logs.
- ☐ Fix production-only problems.
- ☐ Confirm backups, rollback steps, and who has access to secrets.

Done when: the system is stable enough for a controlled first operator test.

Week result: A small real operator group can complete real bookings with monitoring and a rollback plan.

Day 78: Prepare first operator

Friday, 04 December 2026

- ☐ Create the first real operator account.
- ☐ Verify company/bus details.
- ☐ Add routes, buses, seat layouts, and schedules carefully.

Done when: the operator's real trip data is ready for a test booking.

Day 79: Internal real-money test

Saturday, 05 December 2026

- ☐ Use a small controlled booking value.
- ☐ Check payment provider, booking confirmation, ticket, notifications, and payout record.
- ☐ Record exact timestamps and references for troubleshooting.

Done when: one controlled real payment matches across provider and database records.

Day 80: Operator acceptance test

Sunday, 06 December 2026

- ☐ Let the operator test schedule, manifest, manual booking, and earnings screens.
- ☐ Collect only launch-blocking issues first.
- ☐ Fix ownership, money, or seat-state errors before cosmetic items.

Done when: the operator can run the core trip workflow without developer help.

Day 81: Add operator 2

Monday, 07 December 2026

- ☐ Repeat onboarding with a second operator.
- ☐ Check that operator 1 and operator 2 data never mix.
- ☐ Test searches that include schedules from both.

Done when: multi-operator data separation and search work correctly.

Day 82: Add operator 3 / small pilot

Tuesday, 08 December 2026

- ☐ Add a third operator only if the first two are stable.
- ☐ Watch error logs, failed payments, stuck locks, and support messages.
- ☐ Keep traffic small and controlled.

Done when: the system handles a small multi-operator pilot cleanly.

Day 83: Launch checklist

Wednesday, 09 December 2026

- ☐ Confirm database backups.
- ☐ Confirm alert/contact plan for payment or booking failures.
- ☐ Confirm privacy/terms/support pages are ready.
- ☐ Confirm secrets are production-only and not committed to Git.

Done when: technical and operational launch blockers are closed.

Day 84: Soft launch day

Thursday, 10 December 2026

- ☐ Open the service to the agreed small audience.
- ☐ Watch bookings, payments, seat locks, email/SMS delivery, and database health.
- ☐ Keep a written list of issues and fix the highest-risk ones first.

Done when: real users can complete bookings while the team can observe and support the system.

Before you call it live

- ☐ A logged-out visitor cannot open private owner, management, or admin data.
- ☐ Changing an ID in the URL cannot expose another customer's booking or another owner's data.
- ☐ A payment is never confirmed from the browser alone; the server verifies the provider message.
- ☐ Two people cannot confirm the same schedule seat.
- ☐ Expired locks become available automatically.
- ☐ Cancellation/refund rules use the policy saved with the booking.
- ☐ Payment, wallet, payout, and audit changes are safe if the same request is received twice.
- ☐ Production secrets are in Vercel/Railway settings, not Git.
- ☐ Database backup and restore steps are written down and tested.
- ☐ A complete real test booking has been checked against payment-provider and database records.

Target

Controlled soft launch by 10 December 2026. Keep the first operator group small, watch the money/seat flows closely, and expand only after the core flow is stable.